

Identification and optimal control of nonlinear systems using recurrent neural networks and reinforcement learning: An overview



Adolfo Perrusquía*, Wen Yu

Departamento de Control Automático, CINVESTAV-IPN National Polytechnic Institute, Mexico City 07360, Mexico

ARTICLE INFO

Article history:

Received 17 October 2020

Revised 14 December 2020

Accepted 21 January 2021

Available online 27 January 2021

Communicated by Zidong Wang

Keywords:

Recurrent neural networks

Reinforcement learning

Model matching

Modeling error

Riccati equation

Lyapunov equation

ABSTRACT

This paper reviews the identification and optimal control problems using recurrent neural networks and reinforcement learning for nonlinear systems both in discrete- and continuous-time. Since neural networks can approximate any nonlinear function, then is shown that it can approximate a dynamical system using some well-identified elements and different neural structures. Existing methods using Lyapunov and Riccati equations to get the neural weights update rules are reviewed. Optimal control using a linear quadratic regulator formulation or reinforcement learning methods are discussed. We discuss the normalized gradient descent algorithm as core algorithm for the optimal control design using reinforcement learning.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

This article describes the use of principles of recurrent neural networks [1] and reinforcement learning [2,3] to design feedforward and $\mathcal{H}_2$ feedback controllers for discrete- and continuous-time dynamical systems. Neural networks provide an elegant framework for system identification and control design. Neural controllers or neurocontrollers are normally designed as a feedforward-feedback control law, that is, using a model compensation given by any neural network structure and a stabilizing controller [4], for example, the proportional derivative (PD) controller [5]. Determining the feedforward control law for nonlinear systems require complete knowledge of the system dynamics [6], which is often difficult or impossible to have. By contrast, neural networks identifies the nonlinear dynamics online using only state and control measurements.

The nonlinear system under the feedforward control law gives a simple model which can be easily controlled by any feedback control law [7]. Most of feedback control laws are not usually designed to be optimal in the sense of minimizing a predefined cost function. One of the most popular philosophies for designing optimal feedback controllers is in the sense of $\mathcal{H}_2$ control [8,9]. $\mathcal{H}_2$ controllers are normally designed offline by solving Hamilton-Jacobi-

Bellman (HJB) equations [10], for example, in the linear case the HJB equation is a Riccati equation which uses complete knowledge of the system dynamics [11]. Without a feedforward control law, the design of a feedback $\mathcal{H}_2$ control law for nonlinear systems becomes difficult or impossible to find [12].

This article shows that the neural network structure known as *recurrent neural network* (RNN) allows for the system identification of nonlinear systems [13] and design of feedforward control laws [1]. Furthermore, the use of the technique known as *reinforcement learning* (RL) allows for the design of feedback $\mathcal{H}_2$ controllers by solving HJB design equations online and without knowing the full system dynamics. In other words, the RNN identifies the nonlinear dynamics and uses the estimated model for the feedforward control design; meanwhile RL is used to find the feedback $\mathcal{H}_2$ control online according to the new dynamics imposed by the feedforward control law [14].

Neural networks can be classified as feedforward and recurrent nets [15]. Most of neural networks identifiers use feedforward networks, for example, multilayer perceptron (MLP) which are used for the approximation of nonlinear functions in an input-output black box, that is, they are static. The main drawback of these neural networks is that they are sensitive to the training data and do not use any information of the local data structure [16]. On the other hand, recurrent neural networks are dynamic systems, that is, they have memory and incorporate a feedback loop. Hence, they have powerful representation capabilities and can successfully

* Corresponding author.

E-mail address: jperrusquia@ctrl.cinvestav.mx (A. Perrusquía).

overcome the disadvantages of feedforward networks [17,18]. In [19,18], RNNs were used for the identification of nonlinear systems such as the Lorenz system and the engine idle speed control [1]; furthermore they have been applied for the identification of robot manipulators using two different designs of RNN [1].

There exists two kinds of neural networks structures: a *series-parallel model* and a *parallel model*. The series-parallel model ensures that all the signals are bounded if the plant is *bounded input bounded output* (BIBO) stable [19] for both feedforward nets and recurrent nets. The parallel model is mainly used for noisy systems since it avoids problems of bias caused by noise on states measurements. Usually, the update law for the neural weights is obtained through the gradient-descent method and Lyapunov stability analysis. This strong techniques gives a feasible way to obtain the weights update law and stability proof of the neural identifier in closed-loop for both discrete- and continuous-time systems. Nevertheless, the stability analysis of discrete-time systems becomes a hard problem due to the appearance of cross products [20,21]. In [15], a first solution for discrete-time systems was proposed. The key idea was the use of a dead-zone to guarantee boundedness of the neural weights. This idea was used in [22] to obtain a new gradient type update rule using a discrete-time Lyapunov equation.

Most of recurrent neural networks at the literature are in continuous-time [23,24] and have a series-parallel structure due to its simplicity for stability analysis and update law design. These networks use a proposed Hurwitz matrix to compute the kernel solution of a Lyapunov equation and use it as a stable direction of the gradient update law [25]. On the other hand, parallel nets use the kernel solution of a Riccati equation obtained through mathematical manipulations of the time-derivative of the Lyapunov function. The neural network weights are online adapted to minimize the identification error according to the proposed Hurwitz matrix [15,19]. So, the weights are updated according to an optimization procedure. In [1,26], different parallel RNNs were developed for the identification of nonlinear systems and robots manipulators which facilitates the design of a feedforward and feedback control laws.

The neurocontroller [27–29] is designed according to the estimated model of the recurrent net which makes a feedback linearization in the closed-loop system and gives a linear model with a bounded disturbance due to modeling error. The $\mathcal{H}_2$ feedback controller is designed according to the new linear dynamics using the solution of a Riccati equation [11]. Nevertheless, if the disturbance is large enough then the closed-loop performance is degraded. Some solutions were proposed to deal with the modeling error such as sliding-mode or adaptive control compensations [4,6,30]. Some model compensations were proposed in [4] to guarantee reference tracking of robot manipulators in presence of gravity, friction and disturbances terms, however the performance is not optimal [31].

Reinforcement learning [32] offers a direct connection between adaptive control and optimal control [33]. More specifically, reinforcement learning is a class of method that enables the design of adaptive controllers [3,34–36] that learn the solutions of an optimal control problem [37]. Reinforcement learning is inspired in how humans learn by adjusting their actions based on reward or punishment stimuli received from the environment.

There exist several type of reinforcement learning algorithms which can be divided in three big classes: value function iteration, policy iteration, and policy search [3]. In the value function iteration, the reinforcement learning method seeks the optimal value function in an iterative way, the most famous and representable algorithm is Q-learning [38,39]. Policy iteration seeks the optimal control policy by evaluating and improving its control policies [40]; some of the most famous algorithms are Sarsa and critic

methods. Policy search seeks the optimal control policy and optimal value function simultaneously by using an adequate representation; the most famous algorithms are policy gradient and actor-critic methods [41,42]. Each class can be model-based [43] or model-free [44]. A complete overview of reinforcement learning methods can be found [33].

In this paper a critic structure is used. The critic assesses a solution for the value function. The algorithm is supported by two steps: policy evaluation and policy improvement. The policy evaluation step applies the control policy and critique its performance, if the performance is not the desired then the control policy is improved and evaluated again by the critic. The process continues until the optimal control policy is found. One necessary condition for the convergence of the reinforcement learning method is to use a persistent exciting condition or an exploration term. Most of the reinforcement learning algorithms use ε -greedy exploration strategy [45], a Gaussian noise term [41], softmax [2], among others. The main idea is to seek all the states possibilities and control actions which satisfy the control objective and minimize a temporal difference error. Some real applications can be found in [39,45] which uses RL techniques to calculate the optimal contact force between robots and an unknown environment. The results show that the RL algorithms need large learning time to converge to the optimal or near optimal solution because they started learning from scratch.

The $\mathcal{H}_2$ controllers presented in this article are completely model free. These controllers are based on neural networks and merges two main topics in control research: system identification and optimal control [22,46,26]. The identification part is done by a recurrent network scheme (either in discrete-time or continuous-time), where a modular structure is given in how the neural identifier must be constructed using either a single layer or multilayer perceptron models. The control part uses the estimated model to compensate the dynamics of the system and facilitate the $\mathcal{H}_2$ control design using either a model-based or reinforcement learning approach. The communion of both techniques facilitate the control design and the learning time. A detail description of the $\mathcal{H}_2$ control, which in this paper is a linear quadratic regulator (LQR) control, is given and it is emphasized the main problems that arises by using a feedforward neural model to compensate the dynamics of the system.

Throughout this paper, $\mathbb{N}$, $\mathbb{R}$, $\mathbb{R}^n$, $\mathbb{R}^{n \times m}$ denote the spaces of natural numbers, real n -vectors, and real $n \times m$ -matrices, respectively; $\lambda(A)$ denotes the eigenvalues of matrix A ; $\lambda_{\min}(A)$ and $\lambda_{\max}(A)$ denotes the minimum and maximum eigenvalues of matrix A , respectively; the norms $\|A\| = \sqrt{\lambda_{\max}(A^T A)}$ and $\|x\|$ stand for the induced Frobenius and vector Euclidean norms, respectively; $\min(\cdot)$ and $\max(\cdot)$ denote the minimum and maximum operators; $\otimes$ denotes the Kronecker product and $\text{vec}(A)$ defines de matrix stretch of matrix A ; where $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{n \times n}$ and $n, m \in \mathbb{N}$.

2. Discrete-time recurrent neural networks for system identification

Discrete-time neural networks [47,48] are more convenient for real applications because almost all of the control schemes are implemented on digital devices. Also the emerging field of machine learning techniques are developed in discrete-time.

Consider the discrete-time nonlinear system

$$x_{k+1} = f(x_k, u_k), \quad (1)$$

with discrete time index k ; where $x_k \in \mathbb{R}^n$ is the state vector, $u_k \in \mathbb{R}^m$ is the control vector, and $f \in \mathbb{R}^n$ is an unknown nonlinear transition function. This section gives a the design of a discrete recurrent net that identifies the system (1). Two neural models

are used: a RNN without hidden layers and a RNN with hidden layers.

2.1. Discrete-RNN without hidden layers

This kind of recurrent net is an idealistic case because assumes that a single layer perceptron (SLP) is enough to identify the non-linearity of the system dynamics. Consider the simplest recurrent network [15] that models the nonlinear system (1)

$$\hat{x}_{k+1} = A\hat{x}_k + W_k\sigma(x_k) + U_k, \quad (2)$$

where $\hat{x}_k \in \mathbb{R}^n$ is the state of the RNN and $U_k = [u_1, \dots, u_m, 0, \dots, 0]^T \in \mathbb{R}^n$ is the control input. The matrix $A \in \mathbb{R}^{n \times n}$ is a stable Hurwitz matrix, $W_k \in \mathbb{R}^{n \times n}$ are the output weights, $\sigma(\cdot) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the activation function. The elements of $\sigma_i(\cdot)$ can be any stationary, bounded and monotone increasing functions. The RNN (2) has a series-parallel structure, that is, it incorporates real and estimates of the states x_k . Fig. 1 shows the diagram of the discrete recurrent net without hidden layers.

To prove stability of the neural identifier two core ideas are needed: a dead-zone [15] condition and the selection of matrix A . The dead-zone condition is designed between the current state x_k and the next state x_{k+1} ; which gives a feasible direction for the neural weights updating and satisfies the following inequality

$$\|x_{k+1}\| \geq \frac{1}{\beta} \|x_k\|, \quad (3)$$

where $\beta \geq 1$. The gain β can be selected large enough such that the dead-zone almost disappear. The matrix A is selected as a stable matrix, where its eigenvalues are within the interval $-\frac{1}{\beta} < \lambda(A) < 0$ and satisfies the Lyapunov equation

$$A^T P^i A - P^i + Q^i + \left(\frac{1}{\beta} I + A\right)^T P^i \left(\frac{1}{\beta} I + A\right) = 0, \quad (4)$$

for some matrix $Q^i > 0 \in \mathbb{R}^n$ and kernel matrix $P^i > 0 \in \mathbb{R}^n$. The solution of the Lyapunov Eq. (4) can be obtained using [49],

$$\text{vec}(P^i) = \left\{ I - A^T \otimes A^T - \left[\frac{1}{\beta} I + A\right]^T \otimes \left[\frac{1}{\beta} I + A\right] \right\}^{-1} \text{vec}(Q^i). \quad (5)$$

For any identification problem, there exists two possible cases: model matching and modeling error. Model matching case implies that the RNN can identify exactly the nonlinear dynamics (1). On

the other hand, in the modeling error case implies that the RNN cannot identify exactly the nonlinear system (1) and its estimated model possess a bounded modeling error.

2.1.1. Model matching

Consider the model matching case. According to the Stone-Weierstrass theorem [50], the nonlinear model (1) can be written as

$$x_{k+1} = Ax_k + W^* \sigma(x_k) + U_k, \quad (6)$$

for some optimal and unknown weight matrix W^* . Define the identification error as

$$\tilde{x}_k = \hat{x}_k - x_k. \quad (7)$$

The state-transition error dynamics between (2) and (6) is

$$\tilde{x}_{k+1} = A\tilde{x}_k + \tilde{W}_k \sigma(x_k), \quad (8)$$

where $\tilde{W}_k = W_k - W^*$ is the output weights error. Notice that (8) does not contain the control input U_k and only depends on states measurements and its estimates.

2.1.2. Modeling error

In the modeling error case, the model (6) is modified as

$$x_{k+1} = Ax_k + W^* \sigma(x_k) + U_k + \varepsilon_f, \quad (9)$$

where $\varepsilon_f \in \mathbb{R}^n$ is the modeling error defined as

$$\varepsilon_f = f(x_k, u_k) - Ax_k - W^* \sigma(x_k) - U_k. \quad (10)$$

It is assumed that the modeling error is bounded, that is, $\|\varepsilon_f\| \leq \bar{\varepsilon}_f$ for some upper bound $\bar{\varepsilon}_f$. The state-transition error dynamics of (2) and (9) is

$$\tilde{x}_{k+1} = A\tilde{x}_k + \tilde{W}_k \sigma(x_k) + \xi_k. \quad (11)$$

where $\xi_k = -\varepsilon_f$. From Lyapunov stability theory, the following gradient-type update rule is obtained to adjust the neural weights,

$$\tilde{W}_{k+1} = \tilde{W}_k - \eta P^i A \tilde{x}_k \sigma^T(x_k) \quad (12)$$

$$\eta = \begin{cases} \eta_0 \frac{1}{1 + \|P^i A \tilde{x}_k \sigma^T(x_k)\|^2} & \text{if } \|\tilde{x}_{k+1}\| \geq \frac{1}{\beta} \|\tilde{x}_k\| \\ 0 & \text{otherwise,} \end{cases} \quad (13)$$

where $\eta_0 \in (0, 1]$ is the learning rate. Stability analysis of discrete recurrent nets are available [15].

2.2. Discrete RNN with hidden layers

A SLP can hardly identify a true nonlinear system, so multilayer perceptron (MLP) are used to overcome this issue. Fig. 2 shows the diagram of the discrete recurrent net with hidden layers.

The recurrent network with hidden layers that model system (1) is

$$\hat{x}_{k+1} = A\hat{x}_k + W_k \sigma(V_k x_k) + U_k, \quad (14)$$

where $W_k \in \mathbb{R}^{n \times r}$ are the output weights, $V_k \in \mathbb{R}^{l \times n}$ are the weights of the hidden layer, $\sigma(\cdot) : \mathbb{R}^l \rightarrow \mathbb{R}^r$ are the activation functions. The RNN without hidden layers is a particular case of these networks under $V_k = I_{n \times n}$.

2.2.1. Model matching

As in the SLP case, two cases are identified: model matching and modeling error. Consider first the model matching case. From the Stone-Weierstrass theorem, the nonlinear model (1) can be written as

$$x_{k+1} = Ax_k + W^* \sigma(V^* x_k) + U_k, \quad (15)$$

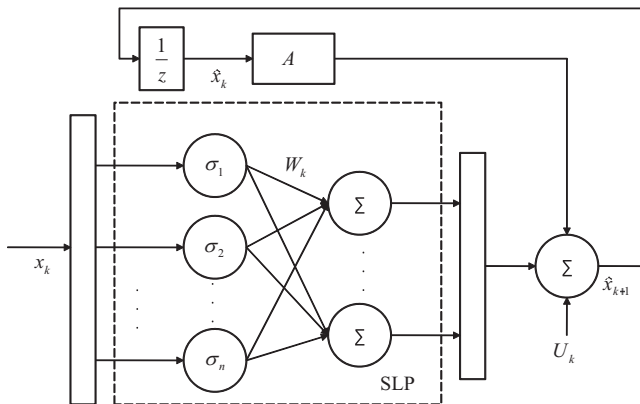


Fig. 1. Discrete-time serial-parallel recurrent neural network without hidden layers. The net is composed of a single layer perceptron (SLP) which uses real measurements of the state to feed the activation functions σ_i . The output of the activation are weighted by an output layer of weights W_k . The state-transition function (2) is given by the sum of the control input U_k , the SLP output and the feedback term given by the unit delay $1/z$ and the Hurwitz matrix A .

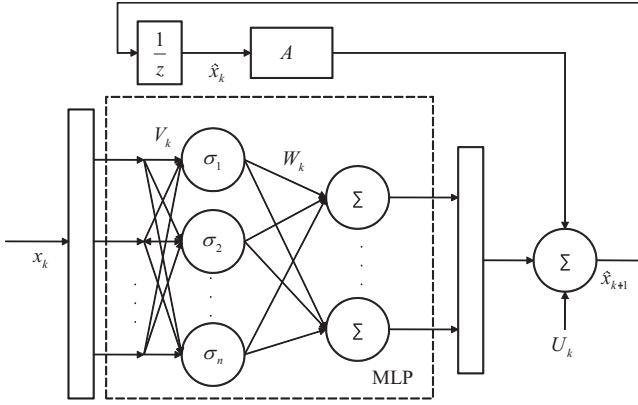


Fig. 2. Discrete-time serial-parallel recurrent neural network with hidden layers. The net is composed of a multilayer perceptron (MLP) which uses real measurements of the state x_k weighted by the hidden weights V_k to feed the activation functions σ_i . The output of the activation function are weighted by an output layer of weights W_k . The state-transition function (14) is given by the sum of the control input U_k , the MLP output and the feedback term given by the unit delay $1/z$ and the Hurwitz matrix A .

where W^* and V^* are optimal and unknown weight matrices.

Taylor series expansion is used to obtain the update rule of the hidden layer weights. In the case of two independent variables, a smooth function $g(x_1, x_2)$ has the next Taylor formula,

$$g(x_1, x_2) = \sum_{k=0}^{l-1} \frac{1}{k!} \left[(x_1 - x_1^0) \frac{\partial}{\partial x_1} + (x_2 - x_2^0) \frac{\partial}{\partial x_2} \right]^k g(x_1^0, x_2^0) + \varepsilon_g,$$

where ε_g is the remainder of the Taylor formula. Let x_1 and x_2 correspond to W^* and V^*x_k , respectively; x_1^0 and x_2^0 correspond to W_k and V_kx_k , respectively; and $l=2$. Using Taylor series around the point W_k and V_kx_k on the function $W^*\sigma(V^*x_k)$ gives,

$$W^*\sigma(V^*x_k) = W_k\sigma(V_kx_k) - \tilde{W}_k\sigma(V_kx_k) - W_k\sigma'V_kx_k - \varepsilon_W,$$

where $\tilde{W}_k = W_k - W^*$ is the hidden layer weights error, ε_W is a second-order approximation error of the Taylor series, and σ' is the derivative of the nonlinear function σ respect to V^*x_k . The identification error dynamics between (14) and (15) under the Taylor series expansion is,

$$\tilde{x}_{k+1} = A\tilde{x}_k + \tilde{W}_k\sigma(V_kx_k) + W_k\sigma'\tilde{V}_kx_k + \varepsilon_W. \quad (16)$$

Notice that approximation error ε_W of (16) is equal to zero because it is assumed that the recurrent net matches the nonlinear system dynamics (1).

2.2.2. Modeling error

In the modeling error case, the nonlinear system (1) can be expressed as the next neural model,

$$x_{k+1} = Ax_k + W^*\sigma(V^*x_k) + U_k + \varepsilon_f, \quad (17)$$

where the modeling error ε_f is

$$\varepsilon_f = f(x_k, u_k) - Ax_k - W^*\sigma(V^*x_k) - U_k. \quad (18)$$

The identification error dynamics between (14) and (17) is

$$\tilde{x}_{k+1} = A\tilde{x}_k + \tilde{W}_k\sigma(V_kx_k) + W_k\sigma'\tilde{V}_kx_k + \zeta_k, \quad (19)$$

where $\zeta_k = \varepsilon_W - \varepsilon_f$. From Lyapunov stability theory [51], the following gradient type update rules are obtained to adjust the neural weights of (14),

$$\tilde{W}_{k+1} = \tilde{W}_k - \eta \left(P^i A \tilde{x}_k + W_k \sigma' \tilde{V}_k x_k \right) \sigma^T(x_k) \quad (20)$$

$$\tilde{V}_{k+1} = \tilde{V}_k - \eta \sigma'^T W_k^T P^i A \tilde{x}_k x_k^T$$

where η satisfies

$$\eta = \begin{cases} \frac{\eta_0}{1 + \|P^i A \sigma(x_k)\|^2 + \|\sigma'^T W_k^T P^i A \tilde{x}_k\|^2} & \text{if } \|\tilde{x}_{k+1}\| \geq \frac{1}{\beta} \|\tilde{x}_k\| \\ 0 & \text{otherwise} \end{cases} \quad (21)$$

Stability analysis of discrete recurrent nets with hidden layers are available [14,15,52].

2.3. Feedforward and Feedback $\mathcal{H}_2$ controls design

The past sections address the identification of nonlinear system (1) using recurrent neural networks without (SLP) or with hidden layers (MLP). This section address the neural control design.

The neural controller [53] is designed to guarantee the tracking of a desired reference $x_d \in \mathbb{R}^n$ which is the solution of the following model,

$$x_{k+1}^d = \varphi(x_k^d), \quad (22)$$

where $\varphi(x_k^d)$ is a smooth function. Define the tracking error as

$$e_k = x_k - x_k^d. \quad (23)$$

The neural representation (17) of nonlinear system (1) can be written as,

$$x_{k+1} = Ax_k + W_k\sigma(V_kx_k) + U_k + \zeta_k, \quad (24)$$

where $\zeta_k = -\tilde{W}_k\sigma(V_kx_k) - W_k\sigma'\tilde{V}_kx_k - \zeta_k$. Notice that for the SLP net (in a model matching case), ζ_k is reduced to $\zeta_k = -\tilde{W}_k\sigma(x_k)$; on the other hand, for a modeling error case $\zeta_k = -\tilde{W}_k\sigma(x_k) - \varepsilon_f$. The closed-loop error dynamics between (22) and (24) is

$$e_{k+1} = Ax_k + W_k\sigma(V_kx_k) + U_k + \zeta_k - \varphi(x_k^d). \quad (25)$$

A natural control design is to compensate the dynamics of the closed-loop error dynamics (25) and impose a new desired dynamics. The neural control U_k is designed in this sense and has the following form,

$$U_k = U_1 + U_2 = \varphi(x_k^d) - Ax_k^d - W_k\sigma(V_kx_k) + U_2, \quad (26)$$

where U_1 is the neural feedforward control which uses the neural modeling (14) as model compensation. U_2 is the $\mathcal{H}_2$ feedback control law. So, the closed-loop error dynamics (25) under the neural control (26) is reduced to,

$$e_{k+1} = Ae_k + U_2 + \zeta_k. \quad (27)$$

The feedback $\mathcal{H}_2$ control is in fact a discrete LQR control [54], and satisfies the next discrete algebraic Riccati equation (DARE),

$$A^T P^c A + Q^c - A^T P^c (R^c + P^c)^{-1} P^c A - P^c = 0, \quad (28)$$

where $Q^c = Q^{c^T} > 0 \in \mathbb{R}^{n \times n}$ and $R^c = R^{c^T} > 0 \in \mathbb{R}^{n \times n}$ are weight matrices; and $P^c = P^{c^T} > 0 \in \mathbb{R}^{n \times n}$ is the solution of the DARE. Hence, the feedback control U_2 is

$$U_2 = -Ke_k, \quad K = -(R^c + P^c)^{-1} P^c A. \quad (29)$$

The solution of (29) can be obtained iteratively using Lyapunov recursions [3,33],

$$P_{k+1}^c = (A - K_k)^T P_k^c (A - K_k) + Q^c + K_k^T R^c K_k.$$

Since $(A - K_k)$ is stable, the recursion converges to the DARE (28) solution for any initial value P_0^c . Notice that for the exact model matching $\zeta_k = 0$, then global asymptotic stability of the $\mathcal{H}_2$ control can be concluded using Lyapunov stability theory [7,55]. On the other hand, uniformly ultimate boundedness (UUB) [56] is concluded in the modeling error case. Here the zone of convergence depends in how large is the modeling error ζ_k . Notice that the $\mathcal{H}_2$ controller (29) does not need information of the real system (1) (only uses states and control measurements). The stability of LQR controllers are available in [14,33].

2.4. $\mathcal{H}_2$ reinforcement learning control

The control (29) assumes that all nonlinear dynamics are compensated by the neural controller U_1 . Nevertheless, the neural control U_1 is sensitive to the neural modeling error ξ_k and affects directly the performance of the $\mathcal{H}_2$ controller (29). To overcome this issue, reinforcement learning (RL) is used in order to take into account the modeling error ξ_k of the neural identifier using state and control measurements along the system trajectories [57].

For a $\mathcal{H}_2$ control design, reinforcement learning uses a similar methodology to the LQR control problem. RL seeks to minimize the next value function [2],

$$\mathcal{V}_k = \sum_{i=k}^{\infty} \gamma^{i-k} (e_i^T Q^c e_i + U_2^T R^c U_2), \quad (30)$$

where $\gamma \in [0, 1)$ is a discount factor that guarantees the convergence of $\mathcal{V}_k$. The value function (30) can be written in the next recursive form [45],

$$\mathcal{V}_k = \gamma \mathcal{V}_{k+1} + e_k^T Q^c e_k + U_2^T R^c U_2. \quad (31)$$

Reinforcement learning uses different kind of approximators to obtain the $\mathcal{H}_2$ control [58,59]. To continue on the subject of neural networks, let consider the following neural network approximator [60,61]

$$\hat{\mathcal{V}}_k = \phi^T(e_k) \theta_k = \phi_k^T \theta_k, \quad (32)$$

where $\theta_k \in \mathbb{R}^p$ is a weight vector, $\phi(\cdot) : \mathbb{R}^n \rightarrow \mathbb{R}^p$ are the activation functions with p neurons at the hidden layer. According to the Stone-Weierstrass theorem [50], the value function (31) can be expressed as,

$$\mathcal{V}_k = \phi_k^T \theta^* + \varepsilon_k, \quad (33)$$

where θ^* is an unknown optimal weight value, $\varepsilon_k = \varepsilon(e_k)$ is the neural approximation error.

Define the Hamiltonian of system (27) under the recursive Eq. (31) as,

$$H(e_k, \theta^*) = r_{k+1} + (\phi_{k+1}^T - \phi_k^T) \theta^* = v_k, \quad (34)$$

where $v_k = \varepsilon(e_k) - \gamma \varepsilon(e_{k+1})$ is the residual error of the NN approximator and $r_{k+1} = e_k^T Q^c e_k + U_2^T R^c U_2$ is the utility function. The Hamiltonian in terms of the NN approximator (32) is,

$$\hat{H}(e_k, \theta_k) = r_{k+1} + (\gamma \phi_{k+1}^T - \phi_k^T) \theta_k = \delta_k, \quad (35)$$

where δ_k is an approximation (also known as temporal difference error) of the residual error v_k . The main goal of reinforcement learning is to minimize the error δ_k . This minimization is given by a quadratic function in terms of δ_k and the normalized gradient descent algorithm (the normalized version serves to maintain bounded activation functions and for stability purposes). The neural weights are adjusted by

$$\theta_{k+1} = \theta_k - \alpha \delta_k \frac{q_k}{(q_k^T q_k + 1)^2}, \quad (36)$$

where $\alpha \in (0, 1]$ is the learning rate and $q_k^T = (\gamma \phi_{k+1}^T - \phi_k^T)$. The $\mathcal{H}_2$ control has the following form,

$$U_2 = -\frac{\gamma}{2} R^{c-1} \nabla \phi^T(e_{k+1}) \theta_k, \quad (37)$$

where $\nabla = \partial / \partial e_{k+1}$. The convergence of the reinforcement learning controller corresponds to how δ_k converges to v_k . Stability and convergence of the RL algorithm under the fulfillment of a persistent exciting (PE) condition is discussed in [14,61,62,22].

3. Continuous-time recurrent neural networks for system identification

Continuous-time recurrent neural networks are a kind of recurrent nets which are more friendly to design and analyze. In contrast with discrete-time recurrent nets, continuous-time recurrent nets makes easier the change of the network structure, for example, the use of both series-parallel and parallel nets, which will be discussed above.

First, consider an unknown continuous-time nonlinear dynamical system

$$\dot{x}_t = f(x_t, u_t), \quad (38)$$

with state $x_t \in \mathbb{R}^n$, control input $u_t \in \mathbb{R}^m$, an equilibrium point at $x = 0$, for example, $f(0) = 0$, and $f(x_t, u_t)$ Lipschitz on a set $\Omega \in \mathbb{R}^n$ that contains the origin [12].

In contrast to discrete-time systems, the design and stabilities proof of serial-parallel and parallel neural networks are facilitated in continuous-time because there are no cross terms in the Lyapunov stability analysis. It is possible to design the continuous-time recurrent neural networks in two cases: without hidden nodes and with hidden nodes as it was described for the discrete-time nets.

3.1. Recurrent neural networks without hidden layers

One of the main advantages of continuous-time recurrent nets is the use of parallel nets to overcome noisy states measurements [19]. Consider the next parallel recurrent neural network structure [63],

$$\dot{\hat{x}}_t = A \hat{x}_t + W_t \sigma(\hat{x}_t) + U_t, \quad (39)$$

where $\hat{x}_t \in \mathbb{R}^n$ is the state of the neural network, $W_t \in \mathbb{R}^{n \times n}$ is the output weights matrix, $A \in \mathbb{R}^{n \times n}$ is a Hurwitz matrix, $U_t = [u_{1,t}, \dots, u_{m,t}, 0, \dots, 0]^T \in \mathbb{R}^n$ is the control action; $\sigma(\cdot) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is assumed to have elements increasing monotonically [18]. The identification error is defined as,

$$\tilde{x}_t = \hat{x}_t - x_t. \quad (40)$$

Fig. 3 shows the diagram of the continuous-time parallel recurrent neural network without hidden layers.

Notice that the term within the activation function $\sigma(\cdot)$ is an estimate of the state x_t , that is, $\hat{x}_t$ and hence, it is not possible to use the same procedure as the discrete RNN. However, it is still possible to identify both the model matching and modeling error cases.

3.1.1. Model matching

Consider that (39) can exactly identify (38). From the Stone-Weierstrass theorem [50], the nonlinear model (38) can be written as

$$\dot{x}_t = A x_t + W^* \sigma(x_t) + U_t, \quad (41)$$

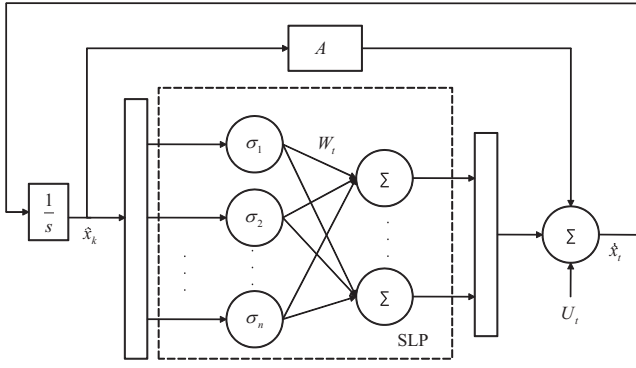


Fig. 3. Continuous recurrent neural network without hidden layers. The net is composed of a single layer perceptron (SLP) which uses estimates of the states $\hat{x}_t$ to feed the activation functions σ_i . The output of the activation function are weighted by an output layer of weights W_t . The RNN dynamics (39) is given by the sum of the control input U_t , the SLP output and the feedback term given by the integrator $1/s$ and the Hurwitz matrix A .

where W^* is an unknown optimal weight matrix. Define the difference between the activation functions as $\tilde{\sigma}_t = \sigma(\hat{x}_t) - \sigma(x_t)$ that satisfies the next inequality,

$$\tilde{\sigma}_t^\top \Lambda \tilde{\sigma}_t \leq \tilde{x}_t^\top \Lambda_\sigma \tilde{x}_t, \quad (42)$$

where Λ and Λ_σ are known normalizing positive definite constant matrices. The identification error dynamics between (39) and (41) is

$$\dot{\tilde{x}}_t = A\tilde{x}_t + \tilde{W}_t \sigma(\hat{x}_t) + W^* \tilde{\sigma}_t. \quad (43)$$

To prove stability of (43), two core ideas are needed: a matrix inequality and a Riccati equation solution. The inequality that is used is

$$X^\top Y + Y^\top X \leq X^\top \Lambda^{-1} X + Y^\top \Lambda Y, \quad (44)$$

where $X, Y \in \mathbb{R}^{n \times m}$ and for any positive defined matrix $\Lambda = \Lambda^\top > 0$. The Riccati equation is similar to the presented in the discrete-time version, where the cost index is modified as

$$\mathcal{V}(\tilde{x}_t) = \int_t^\infty \tilde{x}_\tau^\top S \tilde{x}_\tau e^{t-\tau} d\tau,$$

where $S = S^\top > 0$ is a $n \times n$ positive definite weight matrix and the exponential term within the integral is a discount factor with discount factor of one. The solution of the above cost index satisfies the next differential Riccati equation (DRE),

$$-\dot{P}_t = A^\top P_t + P_t A + Q_t + P_t R_t P_t - P_t,$$

where $Q_t = Q_t^\top = S + \Delta_\sigma > 0 \in \mathbb{R}^{n \times n}$, $R_t = \bar{W} + \Lambda \in \mathbb{R}^{n \times n}$ with $\bar{W} = W^* \Lambda^{-1} W^*$. A sufficient condition to obtain a positive solution of the Riccati equation (45) is to select a stable diagonal matrix A . It is worth mentioning that the matrix A can be freely chosen and can be seen as a tunable parameter of the neural identifier.

3.1.2. Modeling error

In the modeling error case, the dynamics (41) is modified as,

$$\dot{x}_t = Ax_t + W^* \sigma(x_t) + U_t + \varepsilon_f, \quad (46)$$

where $\varepsilon_f \in \mathbb{R}^n$ is the modeling error defined as,

$$\varepsilon_f = f(x_t, u_t) - Ax_t - W_t^* \sigma(x_t) - U_t. \quad (47)$$

The Riccati equation (45) holds for this case since it does not contains information about the modeling error. The identification error dynamics between (46) and (38) is

$$\dot{\tilde{x}}_t = A\tilde{x}_t + \tilde{W}_t \sigma(\hat{x}_t) + W^* \tilde{\sigma}_t + \xi_t. \quad (48)$$

where $\xi_t = -\varepsilon_f$. The update rule for the neural weights of (39) is obtained from Lyapunov stability theory and is given by the next gradient type equation,

$$\dot{W}_t = -K_1 P_t \sigma(\hat{x}_t) \tilde{x}_t^\top, \quad (49)$$

where $K_1 \in \mathbb{R}^{n \times n}$ is a positive definite gain matrix. Stability of continuous-time recurrent net is available [1,19,63].

3.2. Recurrent neural networks with hidden layers

The continuous-time SLP recurrent net (39) is suitable for simple nonlinear systems. To give more power (in computational sense) to the recurrent net for the system identification of more complex nonlinear systems it is mandatory to use hidden layers.

Consider the dynamics of a parallel recurrent net with hidden nodes (MLP) for the identification of the nonlinear system (38),

$$\dot{\hat{x}}_t = A\hat{x}_t + W_t \sigma(V_t \hat{x}_t) + U_t \quad (50)$$

where $W_t \in \mathbb{R}^{n \times r}$ is the output weights matrix, $V_t \in \mathbb{R}^{k \times n}$ is the hidden layer weights matrix, and $\sigma(\cdot) : \mathbb{R}^k \rightarrow \mathbb{R}^r$ are the activation functions. As in discrete-time, the recurrent net (50) is the general case of the recurrent net (39) with $V_t = I_{n \times n}$. The structure of the parallel neural network with hidden layers is shown in Fig. 4.

The Riccati equation (45) holds for the MLP net, where the matrix R_t is modified to $R_t = 2\bar{W} + \Lambda$ [18]. The matrices Q_t and R_t of the Riccati equation (45) are still design parameters and in conjunction with matrix A gives a feasible direction for the neural weights updating thorough the kernel solution P_t .

3.2.1. Model matching

Consider the model matching case. From the Stone-Weierstrass theorem [50], the nonlinear model (38) can be expressed as,

$$\dot{x}_t = Ax_t + W^* \sigma(V^* x_t) + U_t. \quad (51)$$

V^* can be considered as the initial values of the hidden layer weights or an optimal weight value. In contrast with the discrete-time recurrent net with hidden nodes, Taylor series expansion [18] for both W_t and V_t cannot be used as in (16) because the RNN has a parallel structure. So, Taylor series expansion is used only in the function $\sigma(V_t \hat{x}_t)$ around the point $V_t^* \hat{x}_t$ according to the next difference of activation functions,

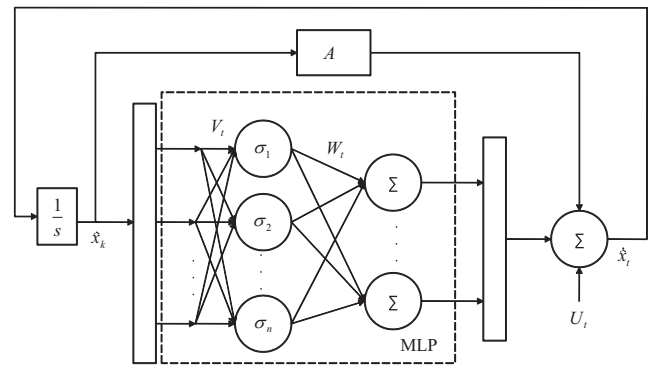


Fig. 4. Continuous recurrent neural network with hidden layers. The net is composed of a multilayer perceptron (MLP) which uses estimates of the states $\hat{x}_t$ weighted by the hidden layer weights V_t to feed the activation functions σ_i . The output of the activation function are weighted by an output layer of weights W_t . The RNN dynamics (50) is given by the sum of the control input U_t , the MLP output and the feedback term given by the integrator $1/s$ and the Hurwitz matrix A .

$$\begin{aligned}\tilde{\sigma}_t &= \sigma(V^* \hat{x}_t) - \sigma(V_t x_t) \\ \sigma(V^* \hat{x}_t) &= \sigma(V_t \hat{x}_t) - \underbrace{(D_\sigma \tilde{V}_t \hat{x}_t + \varepsilon_V)}_{\tilde{\sigma}_t'}\end{aligned}$$

where $\tilde{V}_t = V_t - V^*$, $D_\sigma = \frac{\partial \sigma(Z)}{\partial Z}|_{Z=V_t \hat{x}_t} \in \mathbb{R}^{r \times k}$, ε_V is a second order approximation error, which satisfies

$$\|\varepsilon_V\| \leq L \|\tilde{V}_t \hat{x}_t\|, \quad \|\varepsilon_V\|_\Lambda^2 \leq L \|\tilde{V}_t \hat{x}_t\|_\Lambda^2, \quad L > 0.$$

The identification error dynamics between (51) and (39) is,

$$\dot{\tilde{x}}_t = A \tilde{x}_t + \tilde{W}_t \sigma(V_t \hat{x}_t) + W^* (\tilde{\sigma}_t + \tilde{\sigma}_t'). \quad (52)$$

From (52) is possible to see the main reason of the modification of the weight matrix R_t of the Riccati equation (45) since the optimal weight matrix W^* affects both $\tilde{\sigma}_t$ and $\tilde{\sigma}_t'$.

3.2.2. Modeling error

In the modeling error case, the dynamics (51) is modified as,

$$\dot{x}_t = A x_t + W^* \sigma(V^* x_t) + U_t + \varepsilon_f, \quad (53)$$

where $\varepsilon_f \in \mathbb{R}^n$ is the modeling error defined by

$$\varepsilon_f = f(x_t, u_t) - A x_t - W^* \sigma(V^* x_t) - U_t. \quad (54)$$

The identification error dynamics considering the modeling error ε_f of (54) is,

$$\dot{\tilde{x}}_t = A \tilde{x}_t + \tilde{W}_t \sigma(V_t \hat{x}_t) + W^* (\tilde{\sigma}_t + \tilde{\sigma}_t') + \xi_t. \quad (55)$$

where $\xi_t = -\varepsilon_f$. From Lyapunov stability theory, the following update rules for the neural weights are obtained,

$$\begin{aligned}\dot{W}_t &= -K_1 P_t [\sigma(V_t \hat{x}_t) - D_\sigma \tilde{V}_t \hat{x}_t] \tilde{x}_t^\top \\ \dot{V}_t &= -K_2 P_t W_t D_\sigma \tilde{x}_t \hat{x}_t^\top - \frac{1}{2} K_2 \Lambda \tilde{V}_t \hat{x}_t \hat{x}_t^\top.\end{aligned} \quad (56)$$

for some gain matrices $K_1, K_2 > 0$. Some authors use a dead-zone condition to avoid drift in the neural weights as,

$$\begin{aligned}\dot{W}_t &= s K_1 P_t [\sigma(V_t \hat{x}_t) - D_\sigma \tilde{V}_t \hat{x}_t] \tilde{x}_t^\top \\ \dot{V}_t &= -s K_2 P_t W_t D_\sigma \tilde{x}_t \hat{x}_t^\top - s \frac{1}{2} K_2 \Lambda \tilde{V}_t \hat{x}_t \hat{x}_t^\top,\end{aligned} \quad (57)$$

with

$$s = \begin{cases} 1 & \text{if } \|\tilde{x}_t\| > \sqrt{\frac{\bar{\varepsilon}_f}{\lambda_{\min}(Q_t - P_t - \Lambda \sigma)}}, \\ 0 & \text{otherwise} \end{cases} \quad (58)$$

where $\|\varepsilon_f\| \leq \bar{\varepsilon}_f > 0$ is an upper bound of the modeling error. Stability analysis of the identification error dynamics under either the update rules (55) and (57) are discussed in [1,19,25]

3.3. Feedforward and Feedback $\mathcal{H}_2$ controls design

The neural controller is based on a feedback linearization control scheme which uses the estimated model obtained from the recurrent net as model compensation, then the feedback $\mathcal{H}_2$ is obtained from the remainder dynamics.

The nonlinear neural dynamics (53) can be expressed as,

$$\dot{x}_t = A x_t + W_t \sigma(V_t \hat{x}_t) + U_t + \zeta_t, \quad (59)$$

where $\zeta_t = \varepsilon_f - W^* (\tilde{\sigma}_t + \tilde{\sigma}_t') - \tilde{W}_t \sigma(V_t \hat{x}_t)$ is a modeling error. The neural controller is designed to guarantee tracking of a desired reference $x_d \in \mathbb{R}^n$ which is the solution of the following known nonlinear reference model [19]

$$\dot{x}_{d_t} = \varphi(x_{d_t}) \quad (60)$$

with a fixed and known initial condition. The function $\varphi(\cdot)$ denotes the dynamics of the reference model which is assumed to be smooth. Define the tracking error $e_t \in \mathbb{R}^n$ as

$$e_t = x_t - x_{d_t}. \quad (61)$$

The closed-loop error dynamics between (59) and (60) is

$$\dot{e}_t = A x_t + W_t \sigma(V_t \hat{x}_t) + U_t + \zeta_t - \varphi(x_{d_t}). \quad (62)$$

The neural control U_t is expressed in terms of a feedforward and feedback control laws as

$$\begin{aligned}U_t &= U_1 + U_2 \\ U_1 &= \varphi(x_{d_t}) - A x_d - W_t \sigma(V_t \hat{x}_t),\end{aligned} \quad (63)$$

where U_1 is a feedback linearization controller for (59) and U_2 is the feedback $\mathcal{H}_2$ control. The dynamics of the tracking error (62) under the feedback linearization controller U_1 becomes

$$\dot{e}_t = A e_t + U_2 + \zeta_t. \quad (64)$$

System (64) is a simple linear system with a disturbance. The $\mathcal{H}_2$ control law is designed such that the following discounted cost function [64] is minimized,

$$\mathcal{V}_2 = \int_t^\infty (e_t^\top Q_c e_t + U_{2,t}^\top R_c U_{2,t}) e^{t-\tau} d\tau \quad (65)$$

where $Q_c = Q_c^\top > 0 \in \mathbb{R}^n$ and $R_c = R_c^\top > 0 \in \mathbb{R}^n$ are weight matrices of the tracking error e_t and control U_2 , respectively. The solution of the minimization problem satisfies the next differential Riccati equation [11,44],

$$-\dot{P}_c = A^\top P_c + P_c A + Q_c - P_c R_c^{-1} P_c - P_c,$$

where $P_c = P_c^\top > 0 \in \mathbb{R}^n$ is the solution of the above Riccati equation.

The $\mathcal{H}_2$ control is a LQR controller of the form

$$U_2 = -K e_t = -R_c^{-1} P_c e_t. \quad (67)$$

The stability of the neural controller are available in [19,18]. The optimality of the $\mathcal{H}_2$ control depends in how large is the modeling error ζ_t and if the control gain K of (67) is large enough such that the modeling error effect is attenuated.

3.4. $\mathcal{H}_2$ reinforcement learning control

The design of U_2 does not contain any information of the real system (38) and hence depends completely in the performance of the feedforward control U_1 . If the neural identifier has large modeling error ε_f , then the performance of the neural controller would be poor since it does not contemplate the modeling error for the control design. One way to solve this problem and guarantee and optimal performance is to use reinforcement learning as in the discrete-time case.

The reinforcement learning algorithm find the solution of (65) using the following neural approximator [12,46],

$$\hat{\mathcal{V}}_2 = \theta_t^\top \phi(e_t) = \theta_t^\top \phi, \quad (68)$$

where $\theta_t \in \mathbb{R}^p$ is a weight vector, $\phi(\cdot) : \mathbb{R}^n \rightarrow \mathbb{R}^p$ is the activation function vector with p neurons in the hidden layer. From the Stone-Weistrass theorem [50] the value function (65) can be written as

$$V_2 = \theta^{*T} \phi + \varepsilon(e_t), \quad (69)$$

where θ^* is an optimal weight value, $\varepsilon(e_t)$ is the approximation error. The time derivative of the value function (65) is

$$\dot{V}_2 = -e_t^\top Q_c e_t - U_2^\top R_c U_2,$$

and the time derivative satisfies $\dot{V}_2 = (\partial V_2 / \partial e_t) \dot{e}_t$. The partial derivative of the value function (69) is

$$\frac{\partial V_2}{\partial e_t} = \nabla \phi^\top \theta^* + \nabla \varepsilon(e_t)$$

where $\nabla = \partial / \partial e_t$. Define the Hamiltonian of the problem using the neural approximator (68) as

$$H(e_t, U_2, \theta^*) = \theta^{*\top} (\nabla \phi \dot{e}_t - \phi) + r_t = v_t$$

where $v_t = -\nabla^\top \varepsilon(e_t) \dot{e}_t + \varepsilon(e_t)$ is the residual error of the neural approximator, which is assumed to be bounded, and $r_t = e_t^\top Q_c e_t + U_2^\top R_c U_2$ is the utility function. Consider the approximation of the Hamiltonian using (68) as,

$$\hat{H}(e_t, U_2, \theta_t) = \theta_t^\top (\nabla \phi \dot{e}_t - \phi) + r_t = \delta_t, \quad (70)$$

where δ_t defines the temporal difference error (TD error) of the reinforcement learning method [45,65]. The objective function of the reinforcement learning is to minimize the squared temporal difference error

$$E = \frac{1}{2} \delta_t^2.$$

The normalized gradient descent algorithm [12] is used to ensure boundedness of the weights as,

$$\begin{aligned} \dot{\theta}_t &= -\alpha \delta_t \frac{q_t^\top}{(q_t^\top q_t + 1)^2} \\ q_t &= \nabla \phi \dot{e}_t - \phi, \end{aligned} \quad (71)$$

where $\alpha > 0$ is the learning rate. The $\mathcal{H}_2$ RL controller is obtained by solving the stationary condition $\partial \hat{H} / \partial U_2 = 0$ as,

$$U_2 = -\frac{1}{2} R_c^{-1} \nabla \phi^\top (e_t) \theta_t. \quad (72)$$

Convergence of the neural weights to the optimal solution is achieved under the fulfillment of a persistent exciting condition [12,44,46,56,57].

4. An illustrative example

Consider the following nonlinear system

$$\begin{aligned} \dot{x}_{1,t} &= -15x_{1,t} + 3\text{sign}(x_{2,t}) + u_{1,t} \\ \dot{x}_{2,t} &= -10x_{2,t} + 2\text{sign}(x_{1,t}) + u_{2,t} \end{aligned}$$

where $x_0 = [x_{1,0}, x_{2,0}]^\top = [10, -10]^\top$. The parameters of the neural network are given in Table 1, it is proposed a stable matrix $A = \begin{bmatrix} -15 & 0 \\ 0 & -10 \end{bmatrix}$. The desired reference is given by

$$\dot{x}_{1,d,t} = x_{2,d,t}, \quad \dot{x}_{2,d,t} = \sin(x_{1,d,t})$$

The continuous-time $\mathcal{H}_2$ controllers using the LQR formulation and the RL based controller are compared. It is used the neural estimator (50) with $\hat{x}_0 = [-5, -5]^\top$. The weights are updated by (56), here $K_i = 20I$ ($i = 1, 2$). For the RL NN approximator (71), the learning rate is $\alpha = 0.3$.

Table 1

Neural networks parameters, initial weight and kernel matrices values, and activation functions.

Activation functions	Kernel matrices	State weight matrices	Control weight matrices	NN weights
$\sigma(x_i) = \frac{2}{1+e^{-2x_i}} - 0.5$	$P_i^0 = \begin{bmatrix} 0.1 & 0.05 \\ 0.05 & 0.1 \end{bmatrix}$	$Q_i = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$	$R_i = \begin{bmatrix} 8 & 2 \\ 2 & 8 \end{bmatrix}$	$W_0 = V_0^\top = \begin{bmatrix} 2 & 0.5 & 1 \\ 1 & 2 & 0.5 \end{bmatrix}$
$\phi(e) = [e_1^2 \ e_1 e_2 \ e_2^2]^\top$	$P_c^0 = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}$	$Q_c = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$R_c = \begin{bmatrix} 0.05 & 0 \\ 0 & 0.05 \end{bmatrix}$	$\theta_0 = [0.1 \ 0.5 \ 0.2]^\top$

The tracking results are shown in Figs. 5 and 6. Notice that both methods can follow the desired reference. The LQR solution presents errors at the minimum and maximum points of the desired reference, due to the modeling error. The RL based control has good tracking performance, because the neural identifier and value function approximator are applied.

The above results show that the neural network can identify the nonlinear system by given the stable matrix A . The matrix A gives a feasible direction for the system identification approximation and achieves fast convergence of the kernel matrix P_i of the Riccati equation. A bad design of the functions $\sigma(\cdot)$ and $\phi(\cdot)$ may lead to large identification error and therefore the tracking error increases.

The Lyapunov value functions are quadratic in terms of the identification error $\tilde{x}_t$ and the tracking error e_t with respect to the kernel matrices and neural activation functions. Large values of R_i causes divergence of the kernel matrix P_i because the update (66) has a quadratic term that grows faster than the linear term.

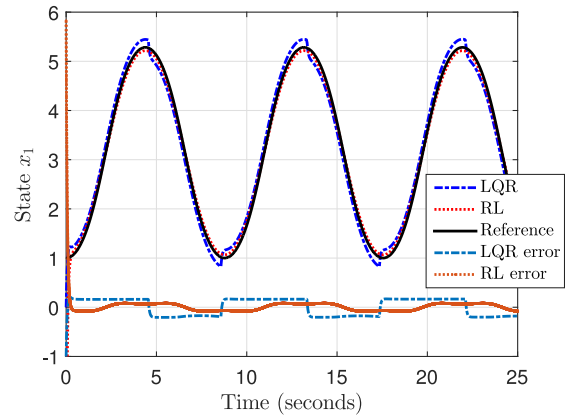


Fig. 5. Tracking of state x_1 . Comparisons between the performance of the LQR and the RL controller in presence of modeling error.

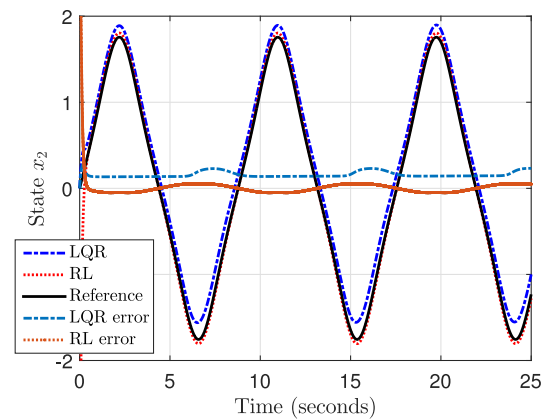


Fig. 6. Tracking of state x_2 . Comparisons between the performance of the LQR and the RL controller in presence of modeling error.

Both LQR and RL solutions present good behavior, however LQR is sensitive to the modeling error. RL solution avoids this problem by taking the modeling error into account at its NN weights update. So, the LQR control is reliable for application where the modeling error does not affect the tracking precision. On the other hand, if the tracking precision depends on modeling accuracy, it is preferable to use the RL controller.

5. Conclusions

This article uses recurrent neural networks and reinforcement learning for the design of an optimal control. Feedforward neural networks can estimate offline the output dynamics of a system but not dynamical systems because they lack of memory. This article describes methods for system identification both in discrete and continuous time using series-parallel and parallel recurrent nets structures. These identifiers estimate the dynamics of a nonlinear system using either a single layer or a multilayer perceptron feedforward net. Both the model matching and modeling error cases are analyzed and discussed.

Optimal controllers need complete knowledge of the system dynamics to compute the solution (if it exists) of a Hamilton–Jacobi–Bellman equations. This article shows that the estimated model provided by the recurrent net can be used to compute a feedforward and H_2 feedback control (which is in fact a LQR controller). Since LQR control is sensitive to modeling error, reinforcement learning is used to robustify the controller. The reinforcement learning method learns online the solution of a Hamilton–Jacobi–Bellman equations using a neural approximation and the feedforward neural controller.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] A.S. Poznyak, E.N. Sanchez, W. Yu, *Differential Neural Networks for Robust Nonlinear Control: Identification, State Estimation and Trajectory Tracking*, World Scientific Publishing Company, 2001.
- [2] R. Sutton, A. Barto, *Reinforcement Learning: An Introduction*, MIT Press, Cambridge, MA, 1998.
- [3] L. Buşoniu, R. Babuka, B. De Schutter, D. Ernst, *Reinforcement Learning and Dynamic Programming using Function Approximators*, CRC Press, 2010.
- [4] A. Perrusquía, W. Yu, Human-in-the-loop control using Euler angles, *Journal of Intelligent & Robotic Systems* 97 (2020) 271–285, <https://doi.org/10.1007/s10846-019-01058-2>.
- [5] A. Perrusquía, J.A. Flores-Campos, C.R. Torres-San-Miguel, A novel tuning method of PD with gravity compensation controller for robot manipulators, *IEEE Access* 8 (2020) 114773–114783, <https://doi.org/10.1109/ACCESS.2020.3003842>.
- [6] W. Yu, A. Perrusquía, Simplified stable admittance control using end-effector orientations, *International Journal of Social Robotics* 12 (5) (2020) 1061–1073, <https://doi.org/10.1007/s12369-019-00579-y>.
- [7] H. Khalil, *Nonlinear Systems*, Prentice Hall, 2002.
- [8] F.L. Lewis, *Optimal Control*, Wiley, New York, NY, USA, 2012.
- [9] L. Xiao, K. Li, M. Duan, Computing time-varying quadratic optimization with finite-time convergence and noise tolerance: a unified framework for zeroing neural network, *IEEE Transactions on Neural Networks and Learning Systems* 30 (11) (2019) 3360–3369, <https://doi.org/10.1109/TNNLS.2019.2891252>.
- [10] A. Al-Tamimi, F. Lewis, M. Abu-Khalaf, Discrete-time nonlinear HJB solution using approximate dynamic programming: convergence proof, *IEEE Transactions on System, Man, and Cybernetics Part B, Cybernetics* 38 (4) (2008) 943–949, <https://doi.org/10.1109/ADPRL.2007.368167>.
- [11] M. Palanisamy, H. Modares, F.L. Lewis, M. Aurangzeb, Continuous-time Q-learning for infinite horizon-discounted cost linear quadratic regulator problems, *IEEE Transactions on Cybernetics* 45 (2) (2015) 165–176, <https://doi.org/10.1109/TCYB.2014.2322116>.
- [12] K. Vamvoudakis, F.L. Lewis, On-line actor-critic algorithm to solve the continuous-time infinite horizon optimal control problem, *Automatica* 46 (2010) 878–888, <https://doi.org/10.1016/j.automatica.2010.02.018>.
- [13] Y. Jiang, J. Fan, T. Chai, F.L. Lewis, Dual-rate operational optimal control for flotation industrial process with unknown operational model, *IEEE Transactions on Industrial Electronics* 66 (6) (2018) 4587–4599, <https://doi.org/10.1109/TIE.2018.2856198>.
- [14] A. Perrusquía, W. Yu, Neural H_2 control using reinforcement learning for unknown nonlinear systems, in: 2020 International Joint Conference on Neural Networks (IJCNN) doi:10.1109/IJCNN48605.2020.9207547.
- [15] W. Yu, Nonlinear system identification using discrete-time recurrent neural networks with stable learning algorithms, *Information Sciences* 158 (2004) 131–147, <https://doi.org/10.1016/j.ins.2003.08.002>.
- [16] W. Yu, X. Li, Some new results on system identification with dynamic neural networks, *IEEE Transactions on Neural Networks* 12 (2) (2001) 412–417, <https://doi.org/10.1109/72.914535>.
- [17] C.-Y. Lee, J.-J. Lee, Adaptive control for uncertain nonlinear systems based on multiple neural networks, *IEEE Transactions on Systems Man and Cybernetics Part B* 34 (1) (2004) 325–333, <https://doi.org/10.1109/TSMCB.2003.811520>.
- [18] W. Yu, Multiple recurrent neural networks for stable adaptive control, *Neurocomputing* 70 (1–3) (2006) 430–444, <https://doi.org/10.1016/j.neucom.2005.12.122>.
- [19] W. Yu, A. Poznyak, Indirect adaptive control via parallel dynamic neural networks, *IEEE Proceedings – Control Theory and Applications* 146 (1) (1999) 25–30, <https://doi.org/10.1049/ip-cta:19990368>.
- [20] K. Narendra, X. Cheng, Adaptive control of discrete-time systems using multiple models, *IEEE Transactions Automatic Control* 45 (9) (1997) 1669–1686, <https://doi.org/10.1109/CDC.1998.761919>.
- [21] S. Batilotti, A. De Santis, Robust output feedback control of nonlinear stochastic systems using neural networks, *IEEE Transactions Neural Networks* 14 (1) (2003) 103–116, <https://doi.org/10.1109/TNN.2002.806609>.
- [22] A. Perrusquía, W. Yu, Discrete-time H_2 neural control using reinforcement learning, *IEEE Transactions on Neural Networks and Learning Systems* (2020) 1–11, <https://doi.org/10.1109/TNNLS.2020.3026010>.
- [23] E. Kosmatopoulos, M. Polycarpou, M. Christodoulou, P. Ioannou, High-order neural network structures for identification of dynamical systems, *IEEE Transactions on Neural Networks* 6 (2) (1995) 422–431, <https://doi.org/10.1109/72.363477>.
- [24] G. Rovothakis, M. Christodoulou, Adaptive control of unknown plants using dynamical neural networks, *IEEE Transactions on System, Man and Cybernetics* 24 (3) (1994) 400–412, <https://doi.org/10.1109/21.278990>.
- [25] W. Yu, X. Li, Recurrent fuzzy neural networks for nonlinear system identification, in: 22nd IEEE International Symposium on Intelligent Control Part of IEEE Multi-conference on Systems and Control, 2007, pp. 590–595, <https://doi.org/10.1109/ISIC.2007.4450952>.
- [26] A. Perrusquía, W. Yu, Neural H_2 control using continuous-time reinforcement learning, *IEEE Transactions on Cybernetics* (2020) 1–10, <https://doi.org/10.1109/TCYB.2020.3028988>.
- [27] J. Martín H., J. de Lope, M. Santos, A method to learn the inverse kinematics of multi-link robots by evolving neuro-controllers, *Neurocomputing* 72 (2009) 2806–2814. doi:10.1016/j.neucom.2008.08.021.
- [28] A. Sahoo, H. Xu, S. Jagannathan, Near optimal event triggered control of nonlinear discrete-time system using neurodynamic programming, *IEEE Transactions on Neural Networks and Learning Systems* 27 (9) (2016) 1801–1815, <https://doi.org/10.1109/TNNLS.2015.2453320>.
- [29] Q. Chen, L. Shi, J. Na, X. Ren, Y. Nan, Adaptive echo state network control for a class of pure-feedback systems with input and output constraints, *Neurocomputing* 275 (31) (2018) 1370–1382, <https://doi.org/10.1016/j.neucom.2017.09.083>.
- [30] Q. Chen, H. Shi, M. Sun, Echo state network-based backstepping adaptive iterative learning control for strict-feedback systems: An error-tracking approach, *IEEE Transactions on Cybernetics* 50 (7) (2020) 3009–3022, <https://doi.org/10.1109/TCYB.2019.2931877>.
- [31] H. Zhang, L. Cui, X. Zhang, Y. Luo, Data-driven robust approximate optimal tracking control for unknown general nonlinear systems using adaptive dynamic programming method, *IEEE Transactions on Neural Networks* 22 (12) (2011) 2226–2236, <https://doi.org/10.1109/TNN.2011.2168538>.
- [32] M. Wiering, M. van Otterlo, *Reinforcement Learning: State-of-Art*, Springer, 2012.
- [33] F.L. Lewis, D. Vrabie, K.G. Vamvoudakis, Reinforcement learning and feedback control using natural decision methods to design optimal adaptive controllers, *IEEE Control Systems Magazine* 32 (6) (2012) 76–105, <https://doi.org/10.1109/MCS.2012.2214134>.
- [34] I. Grondman, L. Buşoniu, R. Babuška, Model learning actor-critic algorithms: performance evaluation in a motion control task, in: 2012 IEEE 51st IEEE Conference on Decision and Control (CDC), IEEE, 2012, pp. 5272–5277, <https://doi.org/10.1109/CDC.2012.6426427>.
- [35] A. Perrusquía, W. Yu, X. Li, Redundant robot control using multi agent reinforcement learning, in: 2020 IEEE 16th International Conference on Automation Science and Engineering (CASE), IEEE, 2020, pp. 1650–1655, <https://doi.org/10.1109/CASE48305.2020.9216774>.
- [36] H. Zhang, D. Liu, Y. Luo, D. Wang, *Adaptive Dynamic Programming for Control*, Springer-Verlag, London, UK, 2013.
- [37] Y. Jiang, J. Fan, T. Chai, J. Li, F.L. Lewis, Data-driven flotation industrial process operational optimal control based on reinforcement learning, *IEEE Transactions on Industrial Informatics* 14 (5) (2017) 1974–1989, <https://doi.org/10.1109/TII.2017.2761852>.
- [38] J.-H. Kim, F. Lewis, Model-free H_∞ control design for unknown linear discrete-time systems via Q-learning with lmi, *Automatica* 46 (2010) 1320–1326, <https://doi.org/10.1016/j.automatica.2010.05.002>.

- [39] A. Perrusquía, W. Yu, A. Soria, Large space dimension reinforcement learning for robot position/force discrete control, in: 2019 6th International Conference on Control, Decision and Information Technologies (CoDIT), IEEE, 2019, pp. 91–96. doi:10.1109/CoDIT.2019.8820575.
- [40] J. Martín H., J. de Lope, D. Maravall, Robust high performance reinforcement learning through weighted k-nearest neighbors, *Neurocomputing* 74 (2011) 1251–1259. doi:10.1016/j.neucom.2010.07.027.
- [41] I. Grondman, L. Buşoniu, G.A. Lopes, R. Babuška, A survey of actor-critic reinforcement learning: standard and natural policy gradients, *IEEE Transactions on Systems, Man, and Cybernetics, PART C* 42 (6) (2012) 1291–1307. <https://doi.org/10.1109/TSMCC.2012.2218595>.
- [42] A. Perrusquía, W. Yu, Continuous-time reinforcement learning for robust control under worst-case uncertainty, *International Journal of Systems Science* (2020) 1–15. <https://doi.org/10.1080/00207721.2020.1839142>.
- [43] R. Kamalapurkar, P. Walters, W. Dixon, Model-based reinforcement learning for approximate optimal regulation, *Automatica* 64 (2016) 94–104. <https://doi.org/10.1016/j.automatica.2015.10.039>.
- [44] H. Modares, F.L. Lewis, Linear quadratic tracking control of partially-unknown continuous-time systems using reinforcement learning, *IEEE Transactions on Automatic Control* 59 (11) (2014) 3051–3056. <https://doi.org/10.1109/TAC.2014.2317301>.
- [45] A. Perrusquía, W. Yu, A. Soria, Position/force control of robot manipulators using reinforcement learning, *Industrial Robot: The International Journal of Robotics Research and Application* 46 (2) (2019) 267–280. <https://doi.org/10.1108/IR-10-2018-0209>.
- [46] D. Vrabie, F.L. Lewis, Neural networks approach for continuous-time direct adaptive optimal control for partially unknown nonlinear systems, *Neural Networks* 22 (2009) 237–246. <https://doi.org/10.1016/j.neunet.2009.03.008>.
- [47] W. Yu, X. Li, Discrete-time neuro identification without robust modification, *IEEE Proceedings Control Theory Applications* 150 (3) (2003) 311–316. <https://doi.org/10.1049/ip-cta:20030204>.
- [48] Y. Jiang, J. Fan, W. Gao, T. Chai, F.L. Lewis, Cooperative adaptive optimal output regulation of nonlinear discrete-time multi-agent systems, *Automatica* 121 (2020). <https://doi.org/10.1016/j.automatica.2020.109149>.
- [49] C. Wang, Y. Li, S. Sam Ge, T. Heng Lee, Optimal critic learning for robot control in time-varying environments, *IEEE Transactions on Neural Networks and Learning Systems* 26 (10) (2015) 2301–2310. <https://doi.org/10.1109/TNNLS.2014.2378812>.
- [50] G. Cybenko, Approximation by superposition of sigmoidal activation function, *Mathematics of Control, Signals, and System* 2 (1989) 303–314. <https://doi.org/10.1007/BF02551274>.
- [51] H. Nounou, K. Passino, Stable auto-tuning of adaptive fuzzy/neural controllers for nonlinear discrete-time systems, *IEEE Transactions on Fuzzy Systems* 12 (1) (2004) 70–83. <https://doi.org/10.1109/TFUZZ.2003.822680>.
- [52] G. Leon, I. Lee, Neural network indirect adaptive control with fast learning algorithm, *Neurocomputing* 13 (2–4) (1996) 185–199. [https://doi.org/10.1016/0925-2312\(95\)00091-7](https://doi.org/10.1016/0925-2312(95)00091-7).
- [53] X. Dong, L. Land Zhong, C. Sun, H. He, Adaptive event-triggered control based on heuristic dynamic programming for nonlinear discrete-time systems, *IEEE Transactions on Neural Networks and Learning Systems* 28 (7) (2017) 1594–1605. doi:10.1109/TNNLS.2016.2541020.
- [54] Q. Xie, B. Luo, F. Tan, Discrete-time lqr optimal tracking control problems using approximate dynamic programming algorithm with disturbance, in: 2013 Fourth International Conference on Intelligent Control and Information Processing (ICICIP), IEEE, 2013, pp. 716–721. <https://doi.org/10.1109/ICICIP.2013.6568166>.
- [55] T. Zhang, S. Ge, C. Hang, Stable adaptive control for a class of nonlinear systems using a modified lyapunov function, *IEEE Transactions on Automatic Control* 45 (2000) 129–132. <https://doi.org/10.1109/9.827369>.
- [56] F.L. Lewis, S. Jagannathan, A. Yeşildirek, *Neural Network Control of Robot Manipulators and Nonlinear Systems*, Taylor & Francis, 1999.
- [57] B. Kiumarsi, K.G. Vamvoudakis, H. Modares, F.L. Lewis, Optimal and autonomous control using reinforcement learning: a survey, *IEEE Transactions on Neural Networks and Learning Systems* 29 (6) (2018) 2042–2062. <https://doi.org/10.1109/TNNLS.2017.2773458>.
- [58] A. Perrusquía, W. Yu, Robust control under worst-case uncertainty for unknown nonlinear systems using modified reinforcement learning, *International Journal of Robust and Nonlinear Control* 30 (7) (2020) 2920–2936. <https://doi.org/10.1002/rnc.4911>.
- [59] A. Perrusquía, W. Yu, X. Li, Multi-agent reinforcement learning for redundant robot control in task-space, *International Journal of Machine Learning and Cybernetics* (2020) 1–11. <https://doi.org/10.1007/s13042-020-01167-7>.
- [60] A. Perrusquía, W. Yu, Robot position/force control in unknown environment using hybrid reinforcement learning, *Cybernetics and Systems* 51 (4) (2020) 542–560. <https://doi.org/10.1080/01969722.2020.1758466>.
- [61] B. Kiumarsi, F.L. Lewis, Actor-critic based optimal tracking for partially unknown nonlinear discrete-time systems, *IEEE Transactions on Neural Networks and Learning Systems* 26 (1) (2015) 140–151. <https://doi.org/10.1109/TNNLS.2014.2358227>.
- [62] B. Kiumarsi, F.L. Lewis, H. Modares, A. Karimpor, M.-B. Naghibi-Sistani, Reinforcement Q-learning for optimal tracking control of linear discrete-time systems with unknown dynamics, *Automatica* 50 (2014) 1167–1175. <https://doi.org/10.1016/j.automatica.2014.02.015>.
- [63] W. Yu, X. Li, Fuzzy identification using fuzzy neural networks with stable learning algorithms, *IEEE Transactions on Fuzzy Systems* 12 (3) (2004) 411–420. <https://doi.org/10.1109/TFUZZ.2004.825067>.
- [64] J. Young Lee, J.B. Park, Y.H. Choi, Integral reinforcement learning for continuous-time input-affine nonlinear systems with simultaneous invariant explorations, *IEEE Transactions on Neural Networks and Learning Systems* 26 (5). doi:10.1109/TNNLS.2014.2328590.
- [65] H. Modares, F.L. Lewis, Z.-P. Jiang, H_∞ tracking control of completely unknown continuous-time systems via off-policy reinforcement learning, *IEEE Transactions on Neural Networks and Learning Systems* 26 (10) (2015) 2550–2562. <https://doi.org/10.1109/TNNLS.2015.2441749>.



Adolfo Perrusquía received the B.S. degree in Mechatronic Engineering from the Interdisciplinary Professional Unit on Engineering and Advanced Technologies of the National Polytechnic Institute (UPIITA-IPN) in 2014, and the M.S. and Ph.D. degrees, both in Automatic Control from the Automatic Control Department at the Center for Research and Advanced Studies of the National Polytechnic Institute (CINVESTAV-IPN) in 2016 and 2020, respectively. His main research of interest focuses on robotics, mechanisms, machine learning, reinforcement learning, nonlinear control, system modeling and system identification.



Wen Yu (Senior Member, IEEE) received the B.S. degree in automatic control from Tsinghua University, Beijing, China, in 1990, and the M.S. and Ph.D. degrees in electrical engineering from Northeastern University, Shenyang, China, in 1992 and 1995, respectively. From 1995 to 1996, he was a Lecturer with the Department of Automatic Control, Northeastern University. Since 1996, he has been with CINVESTAV-IPN (National Polytechnic Institute), Mexico City, Mexico, where he is currently a professor with the Departamento de Control Automático. From 2002 to 2003, he held research positions with the Instituto Mexicano del Petróleo, Mexico City.

He was a Senior Visiting Research Fellow with Queen's University Belfast, Belfast, U.K., from 2006 to 2007, and a Visiting Associate Professor with the University of California at Santa Cruz, Santa Cruz, CA, USA, from 2009 to 2010. He has been holding a visiting professorship at Northeastern University since 2006. Dr. Yu is also a member of the Mexican Academy of Sciences. He serves as an Associate Editor for the IEEE TRANSACTIONS ON CYBERNETICS, Neurocomputing, and Journal of Intelligent & Fuzzy Systems.